

SQL QUERY CHEAT SHEET

A Complete Reference Guide for SQL Queries

Prepared by

MEGAN RUI EN YONG

Student ID: 226280165

Deakin University

Year: 2026

TABLE OF CONTENTS

1. SQL Fundamentals	3
SQL Command Categories	3
2. SELECT Queries.....	4
Basic SELECT Syntax.....	4
SELECT Variations.....	4
3. WHERE Clause & Operators	4
Comparison & Logical Operators	4
LIKE Wildcards	5
4. SQL JOINS	6
Multi-Table JOIN Example.....	6
5. Aggregate Functions & GROUP BY	6
GROUP BY & HAVING	7
6. DDL – Data Definition Language	8
CREATE TABLE.....	8
ALTER TABLE.....	8
Other DDL Commands	8
7. DML – Data Manipulation Language	9
INSERT.....	9
UPDATE	9
DELETE.....	9
8. Subqueries & CTEs.....	10
Subquery Types	10
Common Table Expressions (CTE).....	10
9. Window Functions.....	10
10. Built-in Functions.....	12
String Functions.....	12
Date & Time Functions	12
Numeric & Conditional Functions	13
11. Constraints & Keys.....	14
12. Transactions.....	14
13. SQL Best Practices & Tips.....	15
Performance Tips	15
Security Tips.....	15
Data Integrity Tips	15
Quick Reference: Execution Order.....	15

1. SQL Fundamentals

SQL (Structured Query Language) is the standard language for managing and manipulating relational databases. It allows users to create, read, update, and delete data stored in tables.

SQL Command Categories

Category	Full Name	Commands	Purpose
DDL	Data Definition Language	<code>CREATE</code> , <code>ALTER</code> , <code>DROP</code> , <code>TRUNCATE</code>	Define/modify schema
DML	Data Manipulation Language	<code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code>	Manipulate data
DCL	Data Control Language	<code>GRANT</code> , <code>REVOKE</code>	Control access
TCL	Transaction Control Language	<code>COMMIT</code> , <code>ROLLBACK</code> , <code>SAVEPOINT</code>	Manage transactions
DQL	Data Query Language	<code>SELECT</code>	Query/retrieve data

2. SELECT Queries

Basic SELECT Syntax

```
SELECT column1, column2
FROM table_name
WHERE condition
GROUP BY column
HAVING condition
ORDER BY column ASC|DESC
LIMIT n;
```

SELECT Variations

Query Type	Syntax / Example	Description
All columns	<code>SELECT * FROM employees;</code>	Retrieve every column
Specific cols	<code>SELECT name, salary FROM employees;</code>	Retrieve named columns
Distinct	<code>SELECT DISTINCT dept FROM employees;</code>	Remove duplicate rows
Alias (col)	<code>SELECT name AS FullName FROM emp;</code>	Rename column in output
Alias (table)	<code>SELECT e.name FROM employees e;</code>	Rename table reference
Limit rows	<code>SELECT * FROM emp LIMIT 10;</code>	Return first N rows
Offset	<code>SELECT * FROM emp LIMIT 10 OFFSET 20;</code>	Skip rows, then limit
Computed col	<code>SELECT price * qty AS total FROM orders;</code>	Arithmetic expression

3. WHERE Clause & Operators

Comparison & Logical Operators

Operator	Meaning	Example
=	Equal to	<code>WHERE age = 25</code>
<> or !=	Not equal	<code>WHERE status <> 'inactive'</code>
>	Greater than	<code>WHERE salary > 50000</code>
<	Less than	<code>WHERE age < 30</code>
>=	Greater or equal	<code>WHERE score >= 80</code>
<=	Less or equal	<code>WHERE qty <= 100</code>
BETWEEN	Range (inclusive)	<code>WHERE salary BETWEEN 40000 AND 80000</code>
IN	Match from a list	<code>WHERE dept IN ('HR', 'IT', 'Sales')</code>

Operator	Meaning	Example
NOT IN	Exclude list	WHERE dept NOT IN ('Legal')
LIKE	Pattern match	WHERE name LIKE 'A%'
NOT LIKE	Negate pattern	WHERE name NOT LIKE '%test%'
IS NULL	Check for NULL	WHERE email IS NULL
IS NOT NULL	Check not NULL	WHERE email IS NOT NULL
AND	Both conditions true	WHERE age>18 AND country='AU'
OR	Either condition true	WHERE city='Sydney' OR city='Melbourne'
NOT	Negate condition	WHERE NOT status='deleted'

LIKE Wildcards

Wildcard	Meaning	Example	Matches
%	Zero or more characters	LIKE 'A%'	Anna, Apple, A
%	Ends with pattern	LIKE '%son'	Jackson, Mason
%	Contains pattern	LIKE '%an%'	Mango, Banana
_	Single character	LIKE '_at'	Cat, Bat, Hat
_	Exact length pattern	LIKE 'A__'	Ann, Ali

4. SQL JOINS

JOINS combine rows from two or more tables based on a related column.

JOIN Type	Returns	Syntax Example
INNER JOIN	Rows with matching values in BOTH tables	<code>SELECT * FROM A INNER JOIN B ON A.id = B.id;</code>
LEFT JOIN	All rows from LEFT table + matching right rows (NULL fill)	<code>SELECT * FROM A LEFT JOIN B ON A.id = B.id;</code>
RIGHT JOIN	All rows from RIGHT table + matching left rows (NULL fill)	<code>SELECT * FROM A RIGHT JOIN B ON A.id = B.id;</code>
FULL OUTER JOIN	All rows from both tables (NULL where no match)	<code>SELECT * FROM A FULL OUTER JOIN B ON A.id = B.id;</code>
CROSS JOIN	Cartesian product – every row of A × every row of B	<code>SELECT * FROM A CROSS JOIN B;</code>
SELF JOIN	Join a table to itself	<code>SELECT a.name, b.name FROM emp a JOIN emp b ON a.mgr_id = b.id;</code>

Multi-Table JOIN Example

```
SELECT e.name, d.dept_name, p.project FROM employees e INNER JOIN
departments d ON e.dept_id = d.id LEFT JOIN projects p ON e.id = p.emp_id
WHERE d.dept_name = 'Engineering' ORDER BY e.name;
```

5. Aggregate Functions & GROUP BY

Function	Purpose	Example
<code>COUNT(*)</code>	Count all rows	<code>SELECT COUNT(*) FROM employees;</code>
<code>COUNT(col)</code>	Count non-NULL values	<code>SELECT COUNT(email) FROM employees;</code>
<code>COUNT(DISTINCT col)</code>	Count unique values	<code>SELECT COUNT(DISTINCT dept) FROM employees;</code>
<code>SUM(col)</code>	Sum of values	<code>SELECT SUM(salary) FROM employees;</code>
<code>AVG(col)</code>	Average of values	<code>SELECT AVG(salary) FROM employees;</code>
<code>MIN(col)</code>	Minimum value	<code>SELECT MIN(salary) FROM employees;</code>
<code>MAX(col)</code>	Maximum value	<code>SELECT MAX(salary) FROM employees;</code>
<code>GROUP_CONCAT(col)</code>	Concatenate values in group (MySQL)	<code>SELECT GROUP_CONCAT(name) FROM emp GROUP BY dept;</code>
<code>STDDEV(col)</code>	Standard deviation	<code>SELECT STDDEV(salary) FROM employees;</code>

Function	Purpose	Example
<code>VARIANCE (col)</code>	Variance of values	<code>SELECT VARIANCE(score) FROM students;</code>

GROUP BY & HAVING

```
SELECT dept, COUNT(*) AS headcount, AVG(salary) AS avg_sal FROM employees
WHERE status = 'active' GROUP BY dept HAVING COUNT(*) > 5 ORDER BY avg_sal
DESC;
```

6. DDL – Data Definition Language

CREATE TABLE

```
CREATE TABLE employees (      emp_id      INT      PRIMARY KEY
AUTO_INCREMENT,      name      VARCHAR(100) NOT NULL,      email
VARCHAR(150) UNIQUE NOT NULL,      salary      DECIMAL(10,2) DEFAULT 0.00,
hire_date DATE,      dept_id      INT,      FOREIGN KEY (dept_id) REFERENCES
departments(id) );
```

ALTER TABLE

Operation	Syntax
Add column	<code>ALTER TABLE emp ADD COLUMN phone VARCHAR(20);</code>
Drop column	<code>ALTER TABLE emp DROP COLUMN phone;</code>
Modify column	<code>ALTER TABLE emp MODIFY COLUMN salary FLOAT;</code>
Rename column	<code>ALTER TABLE emp RENAME COLUMN name TO full_name;</code>
Rename table	<code>ALTER TABLE emp RENAME TO employees;</code>
Add constraint	<code>ALTER TABLE emp ADD CONSTRAINT chk_sal CHECK (salary > 0);</code>
Drop constraint	<code>ALTER TABLE emp DROP CONSTRAINT chk_sal;</code>

Other DDL Commands

Command	Syntax / Example	Notes
DROP TABLE	<code>DROP TABLE employees;</code>	Permanently removes table & data
TRUNCATE	<code>TRUNCATE TABLE employees;</code>	Deletes all rows, keeps structure
CREATE INDEX	<code>CREATE INDEX idx_name ON emp(name);</code>	Speeds up queries on column
DROP INDEX	<code>DROP INDEX idx_name ON emp;</code>	Removes index
CREATE VIEW	<code>CREATE VIEW v AS SELECT * FROM emp WHERE active=1;</code>	Virtual table from query
DROP VIEW	<code>DROP VIEW v;</code>	Removes view

7. DML – Data Manipulation Language

INSERT

Type	Syntax
Single row	<code>INSERT INTO employees (name, salary) VALUES ('Alice', 75000);</code>
Multi rows	<code>INSERT INTO employees (name, salary) VALUES ('Bob', 60000), ('Carol', 80000);</code>
From SELECT	<code>INSERT INTO archive SELECT * FROM employees WHERE status='inactive';</code>

UPDATE

Type	Syntax
Single column	<code>UPDATE employees SET salary = 90000 WHERE emp_id = 5;</code>
Multi columns	<code>UPDATE employees SET salary = 90000, title='Senior' WHERE emp_id = 5;</code>
With subquery	<code>UPDATE emp SET dept_id = (SELECT id FROM dept WHERE name='IT') WHERE emp_id=3;</code>
All rows (caution!)	<code>UPDATE employees SET bonus = 500;</code>

DELETE

Type	Syntax
Specific rows	<code>DELETE FROM employees WHERE emp_id = 5;</code>
With condition	<code>DELETE FROM employees WHERE hire_date < '2010-01-01';</code>
All rows (careful)	<code>DELETE FROM employees;</code>
With subquery	<code>DELETE FROM emp WHERE dept_id IN (SELECT id FROM dept WHERE budget < 0);</code>

8. Subqueries & CTEs

Subquery Types

Type	Where Used	Example
Scalar subquery	SELECT / WHERE	<code>SELECT name, (SELECT MAX(salary) FROM emp) AS top_sal FROM emp;</code>
Row subquery	WHERE =	<code>SELECT * FROM emp WHERE (dept_id,job)=(SELECT dept_id,job FROM emp WHERE id=1);</code>
Table subquery	FROM clause	<code>SELECT * FROM (SELECT name, salary FROM emp WHERE salary>50000) AS rich;</code>
Correlated	WHERE (row-by-row)	<code>SELECT * FROM emp e WHERE salary > (SELECT AVG(salary) FROM emp WHERE dept_id=e.dept_id);</code>
EXISTS	WHERE EXISTS	<code>SELECT * FROM dept d WHERE EXISTS (SELECT 1 FROM emp e WHERE e.dept_id=d.id);</code>
NOT EXISTS	WHERE NOT EXISTS	<code>SELECT * FROM dept d WHERE NOT EXISTS (SELECT 1 FROM emp e WHERE e.dept_id=d.id);</code>
IN subquery	WHERE IN	<code>SELECT * FROM emp WHERE dept_id IN (SELECT id FROM dept WHERE budget>100000);</code>

Common Table Expressions (CTE)

```
-- Basic CTE WITH high_earners AS (    SELECT emp_id, name, salary FROM
employees WHERE salary > 80000 ) SELECT h.name, d.dept_name FROM
high_earners h JOIN departments d ON h.dept_id = d.id; -- Recursive CTE
(org chart) WITH RECURSIVE org AS (    SELECT id, name, mgr_id FROM emp
WHERE mgr_id IS NULL      UNION ALL    SELECT e.id, e.name, e.mgr_id FROM
emp e JOIN org o ON e.mgr_id = o.id ) SELECT * FROM org;
```

9. Window Functions

Window functions perform calculations across a set of rows related to the current row, without collapsing them.

Function	Category	Example
<code>ROW_NUMBER()</code>	Ranking	<code>SELECT ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn FROM emp;</code>
<code>RANK()</code>	Ranking	<code>SELECT RANK() OVER (ORDER BY salary DESC) FROM emp;</code>
<code>DENSE_RANK()</code>	Ranking	<code>SELECT DENSE_RANK() OVER (ORDER BY salary DESC) FROM emp;</code>
<code>NTILE(n)</code>	Ranking	<code>SELECT NTILE(4) OVER (ORDER BY salary) AS quartile FROM emp;</code>

Function	Category	Example
<code>LAG(col,n)</code>	Navigation	<code>SELECT LAG(salary,1) OVER (ORDER BY hire_date) AS prev_sal FROM emp;</code>
<code>LEAD(col,n)</code>	Navigation	<code>SELECT LEAD(salary,1) OVER (ORDER BY hire_date) AS next_sal FROM emp;</code>
<code>FIRST_VALUE()</code>	Navigation	<code>SELECT FIRST_VALUE(name) OVER (PARTITION BY dept ORDER BY salary DESC) FROM emp;</code>
<code>LAST_VALUE()</code>	Navigation	<code>SELECT LAST_VALUE(name) OVER (PARTITION BY dept ORDER BY salary DESC) FROM emp;</code>
<code>SUM() OVER()</code>	Aggregate	<code>SELECT SUM(salary) OVER (PARTITION BY dept) AS dept_total FROM emp;</code>
<code>AVG() OVER()</code>	Aggregate	<code>SELECT AVG(salary) OVER (PARTITION BY dept) AS dept_avg FROM emp;</code>
<code>COUNT() OVER()</code>	Aggregate	<code>SELECT COUNT(*) OVER (PARTITION BY dept) AS dept_count FROM emp;</code>

10. Built-in Functions

String Functions

Function	Returns	Example
UPPER(str)	Uppercase string	SELECT UPPER('hello') → 'HELLO'
LOWER(str)	Lowercase string	SELECT LOWER('WORLD') → 'world'
LENGTH(str)	Character count	SELECT LENGTH('SQL') → 3
TRIM(str)	Removes leading/trailing spaces	SELECT TRIM(' hi ') → 'hi'
LTRIM / RTRIM	Trim one side	SELECT LTRIM(' hi')
SUBSTRING(str,pos,len)	Extract part	SELECT SUBSTRING('Deakin',1,3) → 'Dea'
CONCAT(s1,s2,...)	Join strings	SELECT CONCAT('First',' ','Last')
REPLACE(str,old,new)	Replace occurrence	SELECT REPLACE('SQL fun','fun','easy')
INSTR(str,sub)	Position of substring	SELECT INSTR('SQL','Q') → 2
LEFT(str,n)	First n chars	SELECT LEFT('Deakin',3) → 'Dea'
RIGHT(str,n)	Last n chars	SELECT RIGHT('Deakin',3) → 'kin'
LPAD/RPAD(str,len,pad)	Pad string to length	SELECT LPAD('5',3,'0') → '005'
CHAR_LENGTH(str)	Unicode char count	SELECT CHAR_LENGTH('café') → 4
FORMAT(num,dec)	Number as string	SELECT FORMAT(12345.6789,2) → '12,345.68'

Date & Time Functions

Function	Returns	Example
NOW()	Current datetime	SELECT NOW() → '2025-07-14 13:45:00'
CURDATE()	Current date	SELECT CURDATE() → '2025-07-14'
CURTIME()	Current time	SELECT CURTIME() → '13:45:00'
DATE(expr)	Date part only	SELECT DATE(NOW())
YEAR/MONTH/DAY(date)	Extract part	SELECT YEAR(hire_date) FROM emp;
DATEDIFF(d1,d2)	Difference in days	SELECT DATEDIFF(NOW(), hire_date) FROM emp;
DATE_ADD(d, INTERVAL n unit)	Add interval	SELECT DATE_ADD(NOW(), INTERVAL 30 DAY);
DATE_SUB(d, INTERVAL n unit)	Subtract interval	SELECT DATE_SUB(NOW(), INTERVAL 1 YEAR);

Function	Returns	Example
<code>DATE_FORMAT(d,fmt)</code>	Format date string	<code>SELECT DATE_FORMAT(NOW(), '%d-%m-%Y')</code>
<code>TIMESTAMPDIFF(u,d1,d2)</code>	Diff in unit	<code>SELECT TIMESTAMPDIFF(YEAR,hire_date,NOW()) AS yrs;</code>
<code>DAYOFWEEK(date)</code>	Day number (1=Sun)	<code>SELECT DAYOFWEEK('2025-07-14') → 2</code>
<code>WEEKDAY(date)</code>	Weekday (0=Mon)	<code>SELECT WEEKDAY('2025-07-14') → 0</code>

Numeric & Conditional Functions

Function	Returns	Example
<code>ROUND(n,d)</code>	Rounded number	<code>SELECT ROUND(3.14159,2) → 3.14</code>
<code>FLOOR(n)</code>	Round down	<code>SELECT FLOOR(4.9) → 4</code>
<code>CEIL(n)</code>	Round up	<code>SELECT CEIL(4.1) → 5</code>
<code>ABS(n)</code>	Absolute value	<code>SELECT ABS(-42) → 42</code>
<code>MOD(a,b)</code>	Remainder	<code>SELECT MOD(10,3) → 1</code>
<code>POWER(x,y)</code>	x to the power y	<code>SELECT POWER(2,8) → 256</code>
<code>SQRT(n)</code>	Square root	<code>SELECT SQRT(144) → 12</code>
<code>COALESCE(v1,v2,...)</code>	First non-NULL	<code>SELECT COALESCE(NULL,'default') → 'default'</code>
<code>NULLIF(a,b)</code>	NULL if a=b else a	<code>SELECT NULLIF(5,5) → NULL</code>
<code>IFNULL(v, alt)</code>	alt if v is NULL	<code>SELECT IFNULL(NULL,'N/A') → 'N/A'</code>
<code>IF(cond,t,f)</code>	Inline if-else	<code>SELECT IF(salary>70000,'High','Normal') FROM emp;</code>
<code>CASE WHEN</code>	Multi-condition branch	<code>CASE WHEN score>=90 THEN 'A' WHEN score>=80 THEN 'B' ELSE 'C' END</code>

11. Constraints & Keys

Constraint	Purpose	Example
PRIMARY KEY	Unique, not-null row identifier	<code>id INT PRIMARY KEY</code>
FOREIGN KEY	References primary key of another table	<code>FOREIGN KEY (dept_id) REFERENCES dept(id)</code>
UNIQUE	All values in column must be distinct	<code>email VARCHAR(150) UNIQUE</code>
NOT NULL	Column cannot hold NULL	<code>name VARCHAR(100) NOT NULL</code>
DEFAULT	Assigns value when none given	<code>status VARCHAR(20) DEFAULT 'active'</code>
CHECK	Value must satisfy a condition	<code>CHECK (salary >= 0)</code>
AUTO_INCREMENT	Integer auto-increments on insert (MySQL)	<code>id INT AUTO_INCREMENT</code>
COMPOSITE PK	Multiple columns form the primary key	<code>PRIMARY KEY (order_id, product_id)</code>

12. Transactions

Transactions group SQL statements into a single atomic unit ensuring ACID properties.

Property	Meaning
Atomicity	All operations succeed or all are rolled back
Consistency	Database moves from one valid state to another
Isolation	Concurrent transactions do not interfere
Durability	Committed changes persist even after failure

```
START TRANSACTION; UPDATE accounts SET balance = balance - 500 WHERE
acc_id = 1; UPDATE accounts SET balance = balance + 500 WHERE acc_id = 2;
-- If all OK: COMMIT; -- If error: ROLLBACK; -- Partial rollback:
SAVEPOINT sp1; ROLLBACK TO sp1;
```

13. SQL Best Practices & Tips

Performance Tips

- Always use WHERE clause to filter rows; avoid SELECT * in production.
- Create indexes on columns frequently used in WHERE, JOIN, and ORDER BY.
- Use EXPLAIN / EXPLAIN ANALYZE to inspect query execution plans.
- Avoid functions on indexed columns in WHERE clauses (they disable index use).
- Use LIMIT when testing large queries.
- Prefer JOINS over correlated subqueries for better performance.

Security Tips

- Use parameterised queries or prepared statements to prevent SQL injection.
- Limit user privileges to the minimum necessary (principle of least privilege).
- Never concatenate user input directly into SQL strings.
- Use views to restrict data access without exposing base tables.

Data Integrity Tips

- Always use transactions for multi-step operations.
- Define constraints (NOT NULL, CHECK, FK) at the database level.
- Back up data before running UPDATE or DELETE on large tables.
- Test queries with SELECT before converting to UPDATE/DELETE.

Quick Reference: Execution Order

Step	Clause	Description
1	FROM	Identify tables / joins
2	WHERE	Filter rows
3	GROUP BY	Group filtered rows
4	HAVING	Filter groups
5	SELECT	Choose columns
6	DISTINCT	Remove duplicates
7	ORDER BY	Sort results
8	LIMIT/OFFSET	Paginate output